

COMP 8005

Assignment 3

Report

Winston Nguyen
A01297231
Feb 28th, 2026

Purpose	3
Requirements	3
Platforms	3
Language	3
Documents	3
Findings	3
Design	3
Overall Architecture	3
Controller/Worker Responsibilities	4
Controller	4
Worker	4
Communication Protocol	4
Partitioning Strategy	4
Implementation	4
Languages and Libraries	4
Languages	4
Libraries	4
Experimental Results	5
Algorithm Run Times	5
1 Thread	5
Graph	7
2 Threads	7
Graph	9
3 Threads	9
Graph	11
4 Threads	11
Graph	13
10 Thread Prediction	13
10 Threads	13
Graph	15
Analysis	15

Purpose

This program will attempt to crack a password that is hashed using one of the following algorithms: MD5, SHA-256, SHA-512, bcrypt, and yescrypt, and multithreading in the worker and utilizes distributed systems and multiple workers.

Requirements

Task	Status
Crack the password and display results	Fully implemented

Platforms

controller.py and worker.py has been tested on:

- Ubuntu 2023.10

Language

- Python

Documents

- [Design](#)
- [Testing](#)

Findings

Design

Overall Architecture

The architecture of this program is a multithreaded distributed controller/worker architecture, and uses heartbeat protocols.

Controller/Worker Responsibilities

Controller

The controller is responsible for setting up a server, and to parse out the shadow file information from the given user. The controller will parse out the: user, hash algorithm, salts, and the hashed password. Then it will split the work into jobs/chunks that are passed onto different workers that connect with it to crack the password. The controller will also send and receive heartbeat requests/responses to verify each worker's status.

Worker

The worker is responsible for connecting with the server/controller and waiting for a job. After receiving the password information, the worker will then create a given number of threads to use the hash algorithm and password information to use a brute-force method to crack the password, and will return the results back to the controller. The worker will also receive heartbeat requests and give a response with the number of attempts tried.

Communication Protocol

The controller and worker communicate through a TCP socket connection. The controller will start the TCP socket, and await connections. It will have an order of jobs that are to be sent out to workers that connect. The workers will then connect, and await for a response from the server where it will receive the cracking information. After cracking the password, the worker finishes by sending a final response with all the necessary information to the controller.

The controller/worker will also use a heartbeat protocol where the controller will send a request to the worker to verify the worker is both working and to check its status.

Partitioning Strategy

The controller will create chunks based off of start and end indexes to create jobs for workers.

The worker will split the legal charset into the number of threads inputted, splitting up the work evenly.

Implementation

Languages and Libraries

Languages

This program was created using Python 3.13.

Libraries

socket: Create the TCP connection

pickle: Dump the communication information for the connection

argparse: Parse command line arguments

time: Calculates run time lengths

IPAddress: Check if an IP address is valid

bcrypt: Hash algorithm for bcrypt

crypt: Hash algorithm for yescript, SHA-256, SHA-512, MD5

itertools: Iterates through a list of characters

string: Creates list of characters to iterate through

threading: Handles multithreading and locks

queue: Creates a queue of jobs

Experimental Results

Algorithm Run Times

All runs were done using the following password: abc, A2#, 456

Performance

MD5	1 System	2 Systems	3 Systems	5 Systems
Controller Parsing Time	0.0000543	0.0000654	0.0000655	0.0000746
Job Dispatch Latency	0.0001183	0.0001523	0.0001800	0.0002212
Cracking Time	16.4942393	8.2471197	5.4980798	3.2988479
Return Latency	0.4639550	0.3565441	0.2965464	0.2412385
Total end-to-end Runtime	16.9583670	8.5773297	5.7883248	3.5391379

bcrypt	1 System	2 Systems	3 Systems	5 Systems
Controller Parsing Time	0.0000743	0.0000810	0.0000835	0.0000645
Job Dispatch Latency	0.0000983	0.0001300	0.0001600	0.00023423
Cracking Time	263.9936773	131.9968387	87.9978924	52.7987355
Return Latency	0.7201443	0.5543348	0.48646342	0.46463226
Total end-to-end Runtime	264.7139943	132.5470487	88.4781374	53.198633

SHA-256	1 System	2 Systems	3 Systems	5 Systems
Controller Parsing Time	0.0000612	0.0000651	0.0000724	0.0000751
Job Dispatch Latency	0.0001117	0.0001346	0.0001738	0.0002187
Cracking Time	218.1651727	109.0825864	72.7217242	43.6330345
Return Latency	0.6901173	0.5212424	0.4542421	0.3675743
Total end-to-end Runtime	218.8554627	109.6027914	73.1719642	43.9933195

SHA-512	1 System	2 Systems	3 Systems	5 Systems
Controller Parsing Time	0.0000610	0.0000653	0.0000725	0.0000752
Job Dispatch Latency	0.0001232	0.0001272	0.0001812	0.0002242
Cracking Time	271.9724240	135.9862120	90.6574747	54.3944848
Return Latency	1.3116193	1.62638483	0.9643278	0.7576183
Total end-to-end Runtime	273.2842273	136.9864270	91.5577247	55.1447798

yescrypt	1 System	2 Systems	3 Systems	5 Systems
Controller Parsing Time	0.0000647	0.0000725	0.0000754	0.0000821
Job Dispatch Latency	0.0001210	0.0001500	0.0001800	0.0002446
Cracking Time	2231.4369410	1115.7184705	743.8123137	446.2873882
Return Latency	0.2106887	0.2165717	0.7843634	0.7512893
Total end-to-end Runtime	2232.6478153	1116.7186905	744.7125687	447.0376882

Analysis

For every algorithm, it was consistently better with more systems. It wasn't quite linear but more systems did improve the overall end to end runtime. This is mostly due to the fact that the only thing that changes for the controller is sending multiple jobs to multiple systems, but the dispatch latency is negligent and having multiple systems simply allows for more jobs to be run at once.